RAG 前置知识

> 这份文档为零基础读者准备。如果你已经熟悉向量检索、Embedding、分块策略等概念，可以直接翻阅主报告。

一、RAG 到底是什么？用一句话说清楚

RAG = 先搜资料，再写答案。

大模型的知识截止于训练数据最后更新的那一刻。你问它"今天北京的天气怎么样"，它不知道——它上次"上学"是一年前的事了。

RAG 的解决方案：在模型回答之前，先去外部知识库（文档、数据库、搜索引擎）查一下相关资料，然后把查到的资料和问题一起交给模型，让模型基于查到的资料来回答。

```
没有 RAG：用户提问 → 模型凭记忆回答（可能过时/出错）
有 RAG：    用户提问 → 去知识库检索相关内容 → 问题+资料 → 模型基于资料回答
```

二、为什么需要 RAG?

大模型有两个天生的局限：

1. **知识是静态的**。模型训练完成后知识就冻结了。今天发布的新闻、你公司内部的文档、刚更新的代码——模型一概不知。
2. **它会"编造"**。不知道答案时，模型不会说"不知道"，而是会凭空编一个听起来合理的回答——这就是幻觉（Hallucination）。

RAG 同时解决这两个问题：

- 通过检索最新资料，回答可以基于最新信息
- 检索到的资料约束了模型的回答范围，大幅降低幻觉

RAG 的流行不是因为技术多炫酷，而是因为它**让大模型在真实生产环境中变得可用**。

前置概念：预训练（Pre-training）与微调（Fine-tuning）

RAG 属于"不碰模型"的技术路线。要理解这点，需要先知道大模型的训练分成两个阶段：

- **预训练（Pre-training）**：让模型在大规模通用数据（整个互联网的文本）上从头学习语言的基本规律。耗时几个月，花费数千万美元。
- **微调（Fine-tuning）**：在预训练好的模型基础上，用特定任务的数据做针对性训练。成本低得多，只需少量数据和 GPU 时间。

大多数日常使用的大模型（ChatGPT、Claude）都是经过预训练后再微调的。预训练决定了模型的"知识底子"，微调调整它的"行为方式"。RAG 走的是另一条路——不修改模型本身，而是**在模型回答前给它"递小抄"**。

困惑度（Perplexity）

困惑度是衡量大模型"有多确定"的指标。数值越低，模型对预测越自信，质量越高。

一个直观理解：如果模型看到"今天天气真____"时，完全确定下一个词是"好"，困惑度就很低；如果在"好"、"棒"、"热"之间犹豫不决，困惑度就高。

在主报告中，RETRO（7.5B 参数）在困惑度上匹配 GPT-3（175B 参数）——意思是 7.5B 的小模型通过检索外部知识，达到了 175B 大模型的预测质量。

三、RAG 的核心流程：索引 → 检索 → 生成

第一步：索引（Indexing）——把知识存起来

把一堆文档（PDF、网页、数据库记录）处理成模型能快速搜索的格式。

流程：文档 → 分块（chunking）→ 向量化（embedding）→ 存入向量数据库

分块（Chunking）：把长文档切成小段。比如一篇 5000 字的文章，切成 20 段，每段 250 字。段太小会丢失上下文，段太大会引入噪声。

向量化（Embedding）：把每一段文字转换成一个数学向量（几百个数字组成的数组）。语义越相近的文字，它们的向量在数学空间中越靠近。

向量数据库：专门存储和快速搜索向量的数据库。它能在海量向量中快速找到"跟某个查询最像"的那些。

第二步：检索（Retrieval）——找相关资料

用户提问后，把问题同样转成向量，然后去向量数据库里找最相似的几个片段。

第三步：生成（Generation）——基于资料写答案

把用户的问题 + 检索到的相关片段一起交给大模型，让模型基于这些资料生成回答。

四、关键词搜索 vs 语义搜索

两种检索方式，通常配合使用。

方式	原理	优点	缺点
关键词搜索（BM25）	匹配相同的关键词	精确，产品 ID/人名等必须精准匹配的场景好用	不理解同义词，"汽车"搜不到"轿车"
语义搜索（向量）	匹配意思相近的内容	理解语义，"如何减肥"能搜到"控制饮食+运动"	对精确匹配不敏感

2026 年的标准做法是混合搜索（Hybrid Search）：两种方式都跑一遍，然后把结果合并排序。

五、一个完整的 RAG 调用长什么样

```
用户问："公司2024年Q3的营收是多少？"

1. 检索前处理
   → 查询重写："2024年第三季度营收数据"（LLM 把口语化的提问改写成适合检索的格式）

2. 混合检索
   → 向量搜索 + BM25 关键词搜索
   → 各取 top 50，用 RRF 算法合并排序
   → 取 top 10 候选

3. 重排序
   → Cross-encoder 对 top 10 精确打分
   → 取 top 3-5

4. 用检索结果 + 问题生成回答
   → 把 top 3 片段 + 用户问题一起发给 LLM
   → LLM 基于资料回答："2024年Q3营收为12.5亿元..."

5.（可选）溯源
   → 标注每条信息的来源："财务报告第3页第2段"
```

重排序需要用到的两个概念：

Cross-encoder（交叉编码器）：一种成对打分模型——同时看查询和候选文档，直接判断它们的匹配程度。比向量搜索更精确，但速度慢（要逐对计算），所以只用在前者筛选出的 top 候选上。

RRF（Reciprocal Rank Fusion，倒数排名融合）：一种简单有效的排序融合算法。假设向量搜索排第 3 的文档和 BM25 排第 8 的是同一篇，RRF 把两个排名加权合并，让"在两个列表里都排名靠前"的文档胜出。

六、Naive RAG → Advanced RAG → Agentic RAG

RAG 经历了四个阶段的演进：

阶段	思路	比喻
Naive RAG	索引→搜→答，一条直线	去图书馆找一本书，翻到相关页，照着读
Advanced RAG	加优化步骤（查询重写、重排序、混合搜索）	去图书馆前先查目录、找到后让专家确认相关性
Modular RAG	拆成可插拔模块，不同场景组合不同模块	不是只有一个固定设计，而是 LEGO 积木——搭什么造型取决于要建什么
Agentic RAG	模型自主决定搜什么、搜几次、要不要搜	不是只跑一趟图书馆，而是边读边想，发现不够再回去查

Modular RAG 是 Naive → Advanced 之后的自然延伸——不再满足于固定的线性管线，而是把 RAG 拆成可自由组合的模块：线性（Linear）、条件分支（Conditional）、并行（Branching）、循环（Looping）。不同任务场景套用不同流程。

Agentic RAG 是目前最前沿的阶段。Self-RAG 和 CRAG（Corrective RAG）是其代表——模型会自我反思："我刚搜到的资料够用吗？答案有依据吗？不够的话我再搜一轮。"

七、GraphRAG——当 RAG 遇到知识图谱

传统 RAG 适合回答"某份文档里有什么"的问题。但如果问题是"这 1000 份文档综合起来，最核心的主题是什么？"，传统 RAG 就无能为力了——因为它只搜片段，看不到全局。

GraphRAG（微软, 2024） 的解决方案：先把文档构建成知识图谱（提取实体和关系），然后对实体分组生成社区摘要。回答全局性问题时，汇总各个社区的摘要。

代价：索引成本极高（千万级 token 的文档索引成本可能达上万美元）。

LightRAG 是更轻量的替代方案，成本低 6000 倍，支持增量更新。

八、RAG 的常见坑

坑	问题	解决
检索不到	文档切得太碎，关键信息丢了	调大分块，用语义分块
检索到噪音	搜到无关内容干扰模型	加重排序，提高阈值
文档冲突	两份文档说法矛盾	让模型识别矛盾并标注
幻觉仍在	模型不依赖检索结果	用 Self-RAG/CRAG 做事实核查
太慢	检索+重排序+生成延迟累积	缓存常用查询，用 Batch API

九、RAG vs 长上下文——不是二选一

2023-2024 年有一个热门争论：模型上下文窗口已经到 1M+ 了，"直接把整个知识库塞进 prompt"是不是比 RAG 更好？

2026 年的共识：**两者是互补工具，不是竞争对手。**

- **单文档深度分析**（比如分析一份 200 页的合同）→ 长上下文更直接
- **海量知识库问答**（从 100 万份文档中找到相关信息）→ RAG 更高效
- **需要外部最新信息** → RAG 必须
- **高频/低延迟场景** → RAG 更便宜（1M token 的 prompt 成本太高）

混合架构正在成为 2026 年的标准实践：先用 RAG 检索相关段落，再用长上下文模型做综合分析。

十、一张图总结

```
用户提问
    ↓
┌──────────────────────────────────────┐
│            RAG 流水线                 │
│                                      │
│  ① 查询处理（重写/扩展/分解）          │
│                                      │
│  ② 混合检索（向量搜索 + BM25）        │
│      ↓                               │
│  ③ 结果融合（RRF 排序）               │
│      ↓                               │
│  ④ 重排序（Cross-encoder 精确打分）   │
│      ↓                               │
│  ⑤ 生成（LLM 基于检索结果回答）        │
│                                      │
└──────────────────────────────────────┘
    ↓
生成回答（带来源引用）

核心技术栈：
  向量数据库：Pinecone / Milvus / Qdrant / Chroma
  嵌入模型：OpenAI / BGE / Cohere / Qwen embedder
  重排序：Cohere Rerank / BGE-reranker
  框架：LangChain / LlamaIndex / Haystack / Dify
```